

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

REVIEW -0

ASSESMENT -8

BACSE202 LAB COMPONENT PROJECT

B. Tech [CSE & CSE-With Specialization]

AY:2026-27

APPENDIX 1

REAL ESTATE SALES MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

25BBS0002 - HARDIK JAIN



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

CONTENT

S.NO	TITLE	PAGE NO
1	ABSTRACT OF THE PROEJCT	4
2	INTRODUCTION OF THE PROJECT	5
3	BACKGROUD OF THE PROJECT WORK	6
4	HARDWARE AND SOFTWARE REQUIREMENT	7
5	E-R DIAGRAM (MINIMUM 7 ENTITY YOU SHOULD DESIGN)	8
6	DESCRIPTION ABOUT DESIGN OF THE DATABASE	9
7	EXECUTION OF THE SQL QUERY WITH OUTPUT SCREENSHOT TABLE CREATION WITH ALL CONSTRAINTS INSERTION UPTO 5 ROWS DELETION UPDATION (ON EACH TABLE) SELECT SINGLE ROW FUNCTIONS SQL FUCNTIONS GROUP BY HAVING CLAUSE SUB QUERY JOIN QUERY	10 - 28

ABSTRACT

The Real Estate Sales Management System is a relational database application designed to streamline the day-to-day operations of a real estate agency, covering agents, property listings, buyer and seller clients, property showings, purchase offers, and final sale transactions. Manually tracking these interconnected records using spreadsheets or paper files is error-prone and does not scale as an agency's portfolio grows. This project addresses that gap by modelling the domain using seven relational entities - AGENT, PROPERTY, CATEGORY, CLIENT, SHOWING, OFFER and TRANSACTION - connected through primary and foreign key relationships that enforce referential integrity. Each agent is linked to the properties they manage, each property belongs to a category and may be shown to multiple clients, and each accepted offer is followed by a transaction record capturing the final sale price and date. The system was implemented in Oracle Database using SQL*Plus / SQL Developer, and the project demonstrates the complete lifecycle of relational database design and query execution: table creation with primary key, foreign key and check constraints; controlled data insertion; schema evolution through ALTER TABLE statements; data maintenance through UPDATE and DELETE statements, including cascading deletes; and a wide range of retrieval operations including SELECT queries with conditions, single-row functions, aggregate functions, GROUP BY and HAVING clauses, subqueries, and multi-table JOIN operations. The resulting system illustrates how a well-normalised relational schema can support accurate, efficient and consistent management of real estate sales data.

ACKNOWLEDGEMENT

I would like to thank Dr. Mythilli T. (HOD) for reviewing the entire project and guide us for the team leader and also helped me to coordinate my project. After completion of my project, I would like to take this chance to express my sincere gratitude to my project guide Dr Kavitha G. who has guided me thoroughly throughout my project development. I would like to express my gratitude to all our friends and teachers, who helped and cooperated directly or indirectly in the accomplishment of this project. Heartfelt appreciation to those who helped me by cooperative acts, logical device to beautify this project to be effective and user-friendly.

INTRODUCTION

Real estate agencies coordinate a large number of moving parts on a daily basis: agents manage portfolios of properties across different categories, clients view properties through scheduled showings, buyers submit offers that may be accepted, rejected or left pending, and accepted offers eventually mature into completed sale transactions. Each of these activities generates data that must be recorded accurately and made available for quick retrieval - for example, an agent needs to know which of their listings are still active, a manager needs to see which agents are handling the most properties, and an accountant needs a clear record of every completed transaction and its sale price.

When this information is tracked manually, using spreadsheets, paper folders or disconnected documents, several problems emerge. Data duplication becomes common because the same client or property details are re-entered in multiple places. There is no enforced link between related records, so it becomes possible to record an offer for a property that does not exist, or a transaction that refers to a non-existent offer. Updating information such as an agent's office location requires manually finding and editing every occurrence, and deleting a property does not automatically clean up its associated showings, offers, and transactions, leaving behind orphaned records. As the number of properties and clients grows, these manual processes become increasingly slow, inconsistent and unreliable.

A relational database directly addresses these problems. By organising data into well-defined tables connected through primary and foreign keys, a relational database guarantees that related records always refer to valid data, supports cascading updates and deletions where appropriate, and allows complex questions about the data - such as which agent has closed the most sales, or which properties are priced above the average listing price - to be answered with a single, efficient query rather than a manual search.

The purpose of this project is to design and implement a relational database for a Real Estate Sales Management System that models agents, properties, property categories, clients, property showings, offers and completed transactions, and to demonstrate the standard set of SQL operations used to create, populate, modify, and query such a database.

Objectives of the Project

- Design a normalised relational schema consisting of exactly seven entities that accurately represents the real estate sales domain.
- Enforce data integrity using primary key, foreign key, NOT NULL, UNIQUE and CHECK constraints, including cascading delete behaviour where appropriate.
- Populate every table with a representative set of sample records.
- Demonstrate schema evolution using ALTER TABLE statements (adding, renaming, modifying and dropping columns).
- Demonstrate data maintenance using UPDATE and DELETE statements, including the effect of ON DELETE CASCADE.
- Demonstrate data retrieval using SELECT queries, single-row functions, aggregate functions, GROUP BY, HAVING, subqueries and multi-table JOIN operations.

BACKGROUND

A relational database organises data into tables (also called relations), where each table stores records of one type of entity and each column stores one attribute of that entity. Relationships between entities are expressed by storing the primary key of one table as a foreign key in another, which is the mechanism used throughout this project to connect agents, properties, clients, showings, offers and transactions.

Agents and commissions

Every agent is identified by a unique agent_id and is associated with an office (agency_branch) and a commission_rate. Storing agents in a dedicated AGENT table means an agent's details are recorded exactly once and referenced by every property they manage, rather than repeated on every listing.

Property listings and statuses

The PROPERTY table stores each property's address, listing price and current status (Active, Pending or Sold), together with foreign keys to the agent responsible for the listing and the category it belongs to. A CHECK constraint ensures a property can never be listed with a non-positive price.

Property categories

The CATEGORY table separates residential, commercial, luxury, land and industrial listings into distinct, reusable categories, so new categories can be added without altering the structure of the PROPERTY table.

Client tracking

The CLIENT table records buyers, sellers and investors along with their contact phone number, which is constrained to be UNIQUE so that no two client records can share the same phone number.

Showings and appointments

The SHOWING table logs every occasion a client views a property, linking a property_id and a client_id together with the date of the showing.

Offers

The OFFER table records every offer a client makes on a property, including the offer amount and its current status (Accepted, Rejected or Pending), with a CHECK constraint preventing non-positive offer amounts.

Final transactions

The TRANSACTION table records the outcome once an offer is accepted and a sale is finalised, storing the final sale_price and sale_date and linking back to both the property and the accepted offer.

Primary Keys and Foreign Keys

A primary key (PK) is a column, or combination of columns, whose value uniquely

identifies each row in a table; the database automatically rejects duplicate or NULL primary key values, guaranteeing every record can always be uniquely located. A foreign key (FK) is a column whose values must match an existing primary key value in another table, and it is this mechanism that links, for example, every PROPERTY row to a valid AGENT and CATEGORY row. Foreign keys can optionally be declared with ON DELETE CASCADE, which means that deleting a parent row (such as a property) automatically deletes all dependent child rows (its showings, offers and transactions) rather than leaving them pointing at a record that no longer exists.

Data Integrity and Constraint Architecture

The relational schema enforces multi-layered integrity constraints to guarantee domain consistency:
- PRIMARY KEY: Uniquely identifies each record across all seven tables, preventing duplicate or NULL identifiers.
- FOREIGN KEY: Establishes referential integrity across parent-child relations. Tables SHOWING, OFFER, and TRANSACTION enforce ON DELETE CASCADE to automatically remove dependent child records when a property is deleted.
- NOT NULL: Ensures mandatory values for critical fields, including agent names, addresses, and showing dates.
- UNIQUE: Applied to client phone numbers (phone VARCHAR2(15) UNIQUE) to prevent duplicate client profiles.
- CHECK: Enforces domain rules ensuring listing_price, offer_amount, and sale_price remain strictly greater than zero.

Database Normalization (1NF, 2NF, 3NF)

The database design strictly complies with relational normalization standards:
- First Normal Form (1NF): All attributes contain atomic, indivisible values with no repeating groups.
- Second Normal Form (2NF): Eliminates partial functional dependencies. Every non-key attribute depends entirely on the primary key of its relation.
- Third Normal Form (3NF): Eliminates transitive dependencies. For example, property categories are decoupled into a dedicated CATEGORY relation rather than stored redundantly in PROPERTY, preventing update and deletion anomalies.

Transactional Lifecycle and Business Workflow

The operational lifecycle coordinates real estate business processes seamlessly:
1. Listing Management: Agents create property records categorized appropriately with initial status 'Active'.
2. Scheduled Showings: Prospective buyer clients view properties through appointments recorded in SHOWING.
3. Purchase Offers: Buyers submit formal bids tracked in OFFER with statuses 'Pending', 'Accepted', or 'Rejected'.
4. Closing Transactions: An accepted offer matures into a TRANSACTION record capturing the final sale price and date, atomically updating property status to 'Sold' and closing the sales cycle.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above
RAM	4 GB or above
Storage	20 GB free space
Keyboard	Standard Keyboard
Mouse	Standard Mouse
Monitor	Standard Display

Software Requirements

Component	Specification
Operating System	Windows 10 / 11
Database	Oracle Database
SQL Interface	SQL*Plus / Oracle SQL Developer
Language	SQL

Full-Stack Application & Runtime Environment

To complement the Oracle SQL*Plus database engine, a comprehensive full-stack desktop application (EstateFlow) was developed with the following production architecture:

- Desktop GUI Host: Standalone Electron runtime (main.js) managing child processes and native window presentation.
- Frontend Framework: React 18, Vite build tool, React Router 6, Axios HTTP client, and Recharts visualization.
- Backend Architecture: Node.js (v20+), Express.js REST API with modular controllers, services, and route middleware.
- Database Connectivity: Official 'oracledb' driver operating in Thick Mode with Oracle 11g XE client libraries, supporting both password authentication and local Windows OS authentication (SYSDBA).

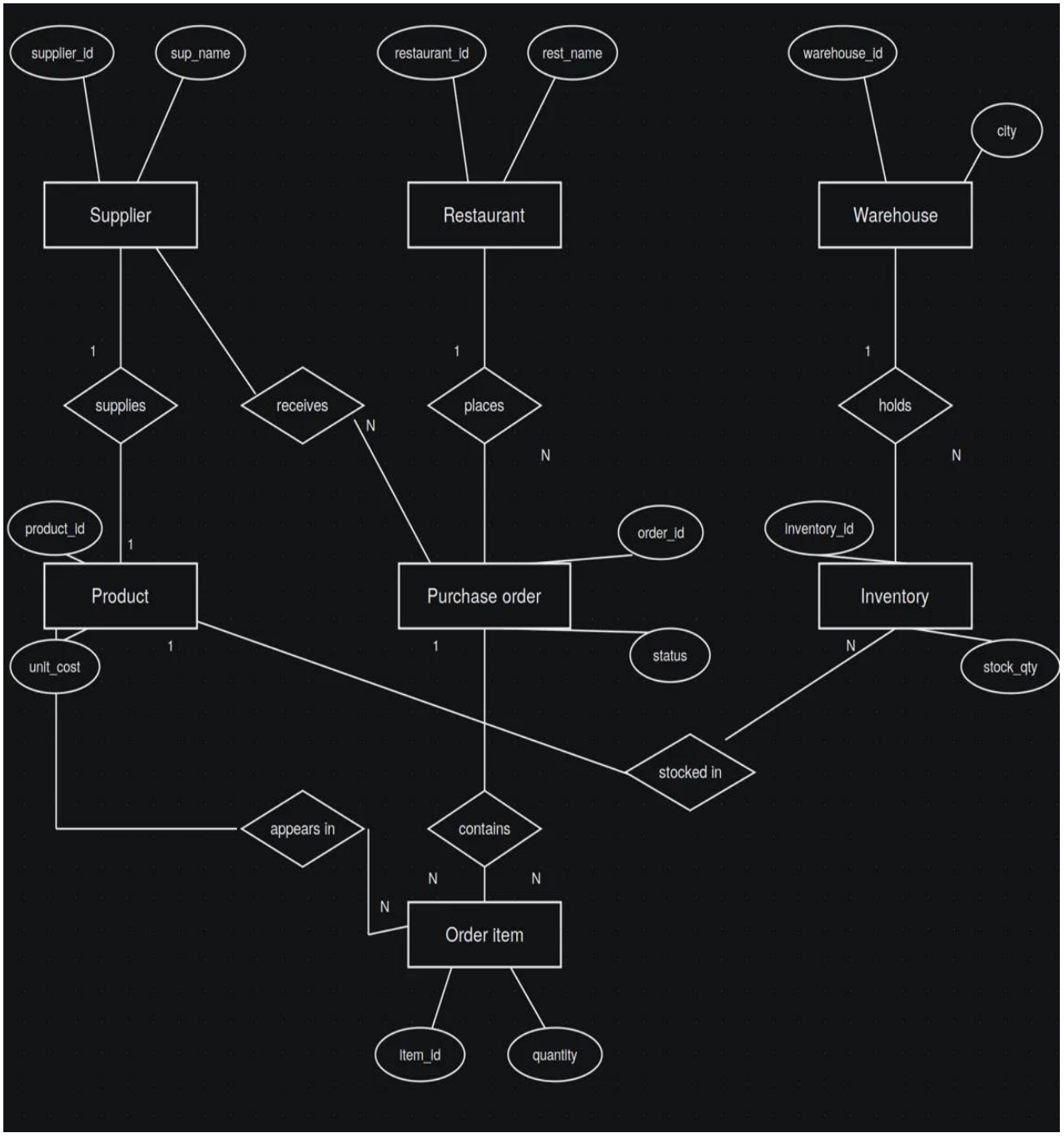
Database Connection Pooling & Security Specifications

- Connection Pool Configuration: Min 2, Max 10 connections, increment step of 2, and 60-second idle timeout, ensuring efficient Oracle session management and fast response times under concurrent operations.
- Role-Based Access Control (RBAC): Database-backed user management via APP_USERS table featuring salted bcrypt password hashing (10 rounds) and stateless JSON Web Tokens (JWT).
- User Entity Roles: Three distinct roles: Admin (unrestricted CRUD & analytical reports), Agent (filtered to manage assigned properties and showings), and Client (personalized offer submission and viewing hub).
- Transaction Management: Explicit COMMIT and automatic ROLLBACK guards across all multi-table operations.

E-R DIAGRAM

The Entity-Relationship diagram below models the seven entities of the Real Estate Sales Management System. Primary keys are underlined and marked (PK); foreign keys are marked (FK). Cardinalities are shown on each connecting line.

Relationship summary: AGENT (1) - PROPERTY (N); CATEGORY (1) - PROPERTY (N); PROPERTY (1) - SHOWING (N); CLIENT (1) - SHOWING (N); PROPERTY (1) - OFFER (N); CLIENT (1) - OFFER (N); PROPERTY (1) - TRANSACTION (1); OFFER (1) - TRANSACTION (1).



DESCRIPTION ABOUT DESIGN OF THE DATABASE

This section describes the structure and purpose of each of the seven tables in the database.

1. AGENT

Stores the real estate agents who manage property listings.

Attribute	Data Type	Constraint	References	Purpose
agent_id	NUMBER(4)	PRIMARY KEY	-	Uniquely identifies each agent
agent_name	VARCHAR2(50)	NOT NULL	-	Full name of the agent
agency_branch	VARCHAR2(50)	NOT NULL	-	Branch/office the agent works from
commission_rate	NUMBER(4,3)	-	-	Commission rate earned on a sale

2. CATEGORY

Classifies properties into residential, commercial, luxury, land or industrial.

Attribute	Data Type	Constraint	References	Purpose
category_id	NUMBER(4)	PRIMARY KEY	-	Uniquely identifies each category
category_name	VARCHAR2(30)	NOT NULL	-	Name of the property category
description	VARCHAR2(100)	-	-	Short description of the category

3. PROPERTY

Stores every property listed by the agency, its price, status, category and agent.

Attribute	Data Type	Constraint	References	Purpose
property_id	VARCHAR2(10)	PRIMARY KEY	-	Uniquely identifies each property
address	VARCHAR2(100)	NOT NULL	-	Street address of the property
listing_price	NUMBER(12,2)	CHECK > 0	-	Listed asking price
status	VARCHAR2(20)	-	-	Active / Pending / Sold
category_id	NUMBER(4)	-	category(category_id)	Category the property belongs to
agent_id	NUMBER(4)	-	agent(agent_id)	Agent managing the listing

4. CLIENT

Stores buyers, sellers and investors who interact with the agency.

Attribute	Data Type	Constraint	References	Purpose
client_id	NUMBER(4)	PRIMARY KEY	-	Uniquely identifies each client

client_name	VARCHAR2(50)	NOT NULL	-	Full name of the client
phone	VARCHAR2(15)	UNIQUE	-	Client contact phone number
client_type	VARCHAR2(20)	-	-	Buyer / Seller / Investor

5. SHOWING

Records each occasion a property is shown to a client.

Attribute	Data Type	Constraint	References	Purpose
showing_id	NUMBER(4)	PRIMARY KEY	-	Uniquely identifies each showing
property_id	VARCHAR2(10)	-	property(property_id) ON DELETE CASCADE	Property being shown
client_id	NUMBER(4)	-	client(client_id)	Client viewing the property
showing_date	DATE	NOT NULL	-	Date the showing took place

6. OFFER

Records every offer made by a client on a property.

Attribute	Data Type	Constraint	References	Purpose
offer_id	NUMBER(4)	PRIMARY KEY	-	Uniquely identifies each offer
property_id	VARCHAR2(10)	-	property(property_id) ON DELETE CASCADE	Property the offer is for
client_id	NUMBER(4)	-	client(client_id)	Client making the offer
offer_amount	NUMBER(12,2)	CHECK > 0	-	Amount offered
offer_status	VARCHAR2(20)	-	-	Accepted / Rejected / Pending

7. TRANSACTION

Records the final completed sale once an offer is accepted.

Attribute	Data Type	Constraint	References	Purpose
transaction_id	NUMBER(4)	PRIMARY KEY	-	Uniquely identifies each transaction
property_id	VARCHAR2(10)	-	property(property_id) ON DELETE CASCADE	Property that was sold
offer_id	NUMBER(4)	-	offer(offer_id)	Accepted offer that led to the sale
sale_price	NUMBER(12,2)	CHECK > 0	-	Final agreed sale price
sale_date	DATE	-	-	Date the sale was completed

EXECUTION OF THE SQL QUERY WITH OUTPUT

This section presents, in logical execution order, the complete set of SQL statements used to create, populate, alter, update, delete and query the database, along with the expected output for each major operation. Spaces are reserved where actual SQL*Plus / SQL Developer execution screenshots can be pasted.

Table Creation with All Constraints

```
CREATE TABLE agent (
   agent_id       NUMBER(4)      PRIMARY KEY,
   agent_name     VARCHAR2(50)   NOT NULL,
   agency_branch  VARCHAR2(50)   NOT NULL,
   commission_rate NUMBER(4,3)
);

CREATE TABLE category (
   category_id    NUMBER(4)      PRIMARY KEY,
   category_name  VARCHAR2(30)   NOT NULL,
   description    VARCHAR2(100)
);

CREATE TABLE property (
   property_id    VARCHAR2(10)   PRIMARY KEY,
   address        VARCHAR2(100)  NOT NULL,
   listing_price  NUMBER(12,2)   CHECK (listing_price > 0),
   status         VARCHAR2(20),
   category_id    NUMBER(4),
   agent_id       NUMBER(4),
   CONSTRAINT fk_property_category FOREIGN KEY (category_id)
      REFERENCES category(category_id),
   CONSTRAINT fk_property_agent FOREIGN KEY (agent_id)
      REFERENCES agent(agent_id)
);

CREATE TABLE client (
   client_id      NUMBER(4)      PRIMARY KEY,
   client_name    VARCHAR2(50)   NOT NULL,
   phone          VARCHAR2(15)   UNIQUE,
   client_type    VARCHAR2(20)
);

CREATE TABLE showing (
   showing_id     NUMBER(4)      PRIMARY KEY,
   property_id    VARCHAR2(10),
   client_id      NUMBER(4),
   showing_date   DATE           NOT NULL,
   CONSTRAINT fk_showing_property FOREIGN KEY (property_id)
      REFERENCES property(property_id) ON DELETE CASCADE,
   CONSTRAINT fk_showing_client FOREIGN KEY (client_id)
      REFERENCES client(client_id)
);

CREATE TABLE offer (
   offer_id       NUMBER(4)      PRIMARY KEY,
   property_id    VARCHAR2(10),
   client_id      NUMBER(4),
```

```
    offer_amount    NUMBER(12,2)    CHECK (offer_amount > 0),
    offer_status    VARCHAR2(20),
    CONSTRAINT fk_offer_property FOREIGN KEY (property_id)
        REFERENCES property(property_id) ON DELETE CASCADE,
    CONSTRAINT fk_offer_client FOREIGN KEY (client_id)
        REFERENCES client(client_id)
);

CREATE TABLE transaction (
    transaction_id  NUMBER(4)       PRIMARY KEY,
    property_id     VARCHAR2(10),
    offer_id        NUMBER(4),
    sale_price      NUMBER(12,2)    CHECK (sale_price > 0),
    sale_date       DATE,
    CONSTRAINT fk_transaction_property FOREIGN KEY (property_id)
        REFERENCES property(property_id) ON DELETE CASCADE,
    CONSTRAINT fk_transaction_offer FOREIGN KEY (offer_id)
        REFERENCES offer(offer_id)
```

Output

```
SQL> CREATE TABLE agent (
  2      agent_id         NUMBER(4)       PRIMARY KEY,
  3      agent_name       VARCHAR2(50)    NOT NULL,
  4      agency_branch    VARCHAR2(50)    NOT NULL,
  5      commission_rate  NUMBER(4,3)
  6  );

Table created.

SQL> CREATE TABLE category (
  2      category_id      NUMBER(4)       PRIMARY KEY,
  3      category_name    VARCHAR2(30)    NOT NULL,
  4      description      VARCHAR2(100)
  5  );

Table created.

SQL> CREATE TABLE property (
  2      property_id      VARCHAR2(10)    PRIMARY KEY,
  3      address          VARCHAR2(100)   NOT NULL,
  4      listing_price    NUMBER(12,2)    CHECK (listing_price > 0),
  5      status           VARCHAR2(20),
  6      category_id      NUMBER(4),
  7      agent_id         NUMBER(4),
  8      CONSTRAINT fk_property_category FOREIGN KEY (category_id)
  9          REFERENCES category(category_id),
 10      CONSTRAINT fk_property_agent FOREIGN KEY (agent_id)
 11          REFERENCES agent(agent_id)
 12  );

Table created.
```

```
SQL> CREATE TABLE client (
  2      client_id       NUMBER(4)       PRIMARY KEY,
  3      client_name     VARCHAR2(50)    NOT NULL,
  4      phone           VARCHAR2(15)    UNIQUE,
  5      client_type     VARCHAR2(20)
  6  );

Table created.

SQL> CREATE TABLE showing (
  2      showing_id      NUMBER(4)       PRIMARY KEY,
  3      property_id     VARCHAR2(10),
  4      client_id       NUMBER(4),
  5      showing_date    DATE            NOT NULL,
  6      CONSTRAINT fk_showing_property FOREIGN KEY (property_id)
  7          REFERENCES property(property_id) ON DELETE CASCADE,
  8      CONSTRAINT fk_showing_client FOREIGN KEY (client_id)
  9          REFERENCES client(client_id)
 10  );

Table created.

SQL> CREATE TABLE offer (
  2      offer_id        NUMBER(4)       PRIMARY KEY,
  3      property_id     VARCHAR2(10),
  4      client_id       NUMBER(4),
  5      offer_amount    NUMBER(12,2)    CHECK (offer_amount > 0),
  6      offer_status    VARCHAR2(20),
  7      CONSTRAINT fk_offer_property FOREIGN KEY (property_id)
  8          REFERENCES property(property_id) ON DELETE CASCADE,
  9      CONSTRAINT fk_offer_client FOREIGN KEY (client_id)
 10          REFERENCES client(client_id)
 11  );

SQL> CREATE TABLE transaction (
  2      transaction_id  NUMBER(4)       PRIMARY KEY,
  3      property_id     VARCHAR2(10),
  4      offer_id        NUMBER(4),
  5      sale_price      NUMBER(12,2)    CHECK (sale_price > 0),
  6      sale_date       DATE,
  7      CONSTRAINT fk_transaction_property FOREIGN KEY (property_id)
  8          REFERENCES property(property_id) ON DELETE CASCADE,
  9      CONSTRAINT fk_transaction_offer FOREIGN KEY (offer_id)
 10          REFERENCES offer(offer_id)
 11  );

Table created.
```

2. Insertion of 5 Rows into Each Table

```
-- AGENT
INSERT INTO agent VALUES (9001, 'Sarah Jenkins', 'Downtown Luxury', 0.030);
INSERT INTO agent VALUES (9002, 'David Miller', 'Westside Suburban', 0.025);
INSERT INTO agent VALUES (9003, 'Michael Chang', 'Coastal Prime', 0.035);
INSERT INTO agent VALUES (9004, 'Amanda Ross', 'Downtown Luxury', 0.030);
INSERT INTO agent VALUES (9005, 'Robert Vance', 'East Valley', 0.020);

-- CATEGORY
INSERT INTO category VALUES (101, 'Residential', 'Single family homes and townhouses');
INSERT INTO category VALUES (102, 'Commercial', 'Office spaces and retail');
INSERT INTO category VALUES (103, 'Luxury', 'High-end premium real estate');
INSERT INTO category VALUES (104, 'Land', 'Empty plots for development');
INSERT INTO category VALUES (105, 'Industrial', 'Warehouses and factories');

-- PROPERTY
INSERT INTO property VALUES ('P5501', '123 Maple St', 450000.00, 'Sold', 101, 9002);
INSERT INTO property VALUES ('P5502', '789 Ocean Blvd', 1250000.00, 'Active', 103, 9003);
INSERT INTO property VALUES ('P5503', '456 Pine Ave', 380000.00, 'Pending', 101, 9001);
INSERT INTO property VALUES ('P5504', '222 Palm Dr', 2100000.00, 'Sold', 103, 9003);
INSERT INTO property VALUES ('P5505', '555 Cedar Rd', 620000.00, 'Active', 101, 9004);

-- CLIENT
INSERT INTO client VALUES (2001, 'John Smith', '9123456780', 'Buyer');
INSERT INTO client VALUES (2002, 'Emily Davis', '9123456781', 'Seller');
INSERT INTO client VALUES (2003, 'Mark Johnson', '9123456782', 'Buyer');
INSERT INTO client VALUES (2004, 'Sophia Brown', '9123456783', 'Investor');
INSERT INTO client VALUES (2005, 'James Wilson', '9123456784', 'Buyer');

-- SHOWING
INSERT INTO showing VALUES (3001, 'P5502', 2001, TO_DATE('10-AUG-2026','DD-MON-YYYY'));
INSERT INTO showing VALUES (3002, 'P5503', 2003, TO_DATE('11-AUG-2026','DD-MON-YYYY'));
INSERT INTO showing VALUES (3003, 'P5505', 2005, TO_DATE('12-AUG-2026','DD-MON-YYYY'));
INSERT INTO showing VALUES (3004, 'P5502', 2004, TO_DATE('13-AUG-2026','DD-MON-YYYY'));
INSERT INTO showing VALUES (3005, 'P5505', 2001, TO_DATE('14-AUG-2026','DD-MON-YYYY'));

-- OFFER
INSERT INTO offer VALUES (4001, 'P5502', 2001, 1200000.00, 'Rejected');
INSERT INTO offer VALUES (4002, 'P5503', 2003, 380000.00, 'Accepted');
INSERT INTO offer VALUES (4003, 'P5505', 2005, 600000.00, 'Pending');
INSERT INTO offer VALUES (4004, 'P5502', 2004, 1225000.00, 'Pending');
INSERT INTO offer VALUES (4005, 'P5501', 2001, 450000.00, 'Accepted');

-- TRANSACTION
INSERT INTO transaction VALUES (5001, 'P5501', 4005, 450000.00, TO_DATE('01-AUG-2026','DD-MON-
YYYY'));
INSERT INTO transaction VALUES (5002, 'P5504', 4002, 2100000.00, TO_DATE('05-AUG-2026','DD-
MON-YYYY'));
INSERT INTO transaction VALUES (5003, 'P5503', 4002, 380000.00, TO_DATE('15-AUG-2026','DD-MON-
YYYY'));
INSERT INTO transaction VALUES (5004, 'P5502', 4004, 1225000.00, TO_DATE('20-AUG-2026','DD-
MON-YYYY'));

INSERT INTO transaction VALUES (5005, 'P5505', 4003, 600000.00, TO_DATE('25-AUG-2026','DD-MON-
YYYY'));
```

Output:

```
SQL> INSERT INTO agent VALUES (9001, 'Sarah Jenkins', 'Downtown Luxury', 0.030);

1 row created.

SQL> INSERT INTO agent VALUES (9002, 'David Miller', 'Westside Suburban', 0.025);

1 row created.

SQL> INSERT INTO agent VALUES (9003, 'Michael Chang', 'Coastal Prime', 0.035);

1 row created.

SQL> INSERT INTO agent VALUES (9004, 'Amanda Ross', 'Downtown Luxury', 0.030);

1 row created.

SQL> INSERT INTO agent VALUES (9005, 'Robert Vance', 'East Valley', 0.020);

1 row created.
SQL> INSERT INTO category VALUES (101, 'Residential', 'Single family homes and townhouses');

1 row created.

SQL> INSERT INTO category VALUES (102, 'Commercial', 'Office spaces and retail');

1 row created.

SQL> INSERT INTO category VALUES (103, 'Luxury', 'High-end premium real estate');

1 row created.

SQL> INSERT INTO category VALUES (104, 'Land', 'Empty plots for development');

1 row created.

SQL> INSERT INTO category VALUES (105, 'Industrial', 'Warehouses and factories');

1 row created.
SQL> INSERT INTO property VALUES ('P5501', '123 Maple St', 450000.00, 'Sold', 101, 9002);

1 row created.

SQL> INSERT INTO property VALUES ('P5502', '789 Ocean Blvd', 1250000.00, 'Active', 103, 9003);

1 row created.

SQL> INSERT INTO property VALUES ('P5503', '456 Pine Ave', 380000.00, 'Pending', 101, 9001);

1 row created.

SQL> INSERT INTO property VALUES ('P5504', '222 Palm Dr', 2100000.00, 'Sold', 103, 9003);

1 row created.

SQL> INSERT INTO property VALUES ('P5505', '555 Cedar Rd', 620000.00, 'Active', 101, 9004);

1 row created.
SQL> INSERT INTO client VALUES (2001, 'John Smith', '9123456780', 'Buyer');

1 row created.

SQL> INSERT INTO client VALUES (2002, 'Emily Davis', '9123456781', 'Seller');

1 row created.

SQL> INSERT INTO client VALUES (2003, 'Mark Johnson', '9123456782', 'Buyer');

1 row created.

SQL> INSERT INTO client VALUES (2004, 'Sophia Brown', '9123456783', 'Investor');

1 row created.

SQL> INSERT INTO client VALUES (2005, 'James Wilson', '9123456784', 'Buyer');

1 row created.
```

```
SQL> INSERT INTO showing VALUES (3001, 'P5502', 2001, TO_DATE('10-AUG-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO showing VALUES (3002, 'P5503', 2003, TO_DATE('11-AUG-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO showing VALUES (3003, 'P5505', 2005, TO_DATE('12-AUG-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO showing VALUES (3004, 'P5502', 2004, TO_DATE('13-AUG-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO showing VALUES (3005, 'P5505', 2001, TO_DATE('14-AUG-2026','DD-MON-YYYY'));

1 row created.
```

```
SQL> INSERT INTO offer VALUES (4001, 'P5502', 2001, 1200000.00, 'Rejected');

1 row created.

SQL> INSERT INTO offer VALUES (4002, 'P5503', 2003, 380000.00, 'Accepted');

1 row created.

SQL> INSERT INTO offer VALUES (4003, 'P5505', 2005, 600000.00, 'Pending');

1 row created.

SQL> INSERT INTO offer VALUES (4004, 'P5502', 2004, 1225000.00, 'Pending');

1 row created.

SQL> INSERT INTO offer VALUES (4005, 'P5501', 2001, 450000.00, 'Accepted');

1 row created.
```

```
SQL> INSERT INTO transaction VALUES (5001, 'P5501', 4005, 450000.00, TO_DATE('01-AUG-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO transaction VALUES (5002, 'P5504', 4002, 2100000.00, TO_DATE('05-AUG-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO transaction VALUES (5003, 'P5503', 4002, 380000.00, TO_DATE('15-AUG-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO transaction VALUES (5004, 'P5502', 4004, 1225000.00, TO_DATE('20-AUG-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO transaction VALUES (5005, 'P5505', 4003, 600000.00, TO_DATE('25-AUG-2026','DD-MON-YYYY'));

1 row created.
```

3. ALTER TABLE Queries

-- Add a new column square_feet to the property table
ALTER TABLE property ADD (square_feet NUMBER);

```
SQL> ALTER TABLE property ADD (square_feet NUMBER);

Table altered.
```

-- Rename agency_branch to office_name in the agent table
ALTER TABLE agent RENAME COLUMN agency_branch TO office_name;

```
SQL> ALTER TABLE agent RENAME COLUMN agency_branch TO office_name;

Table altered.
```

-- Modify listing_price column of property to larger precision
ALTER TABLE property MODIFY (listing_price NUMBER(14,2));

```
SQL> ALTER TABLE property MODIFY (listing_price NUMBER(14,2));

Table altered.
```

-- Drop the commission_rate column from the agent table
ALTER TABLE agent DROP COLUMN commission_rate;

```
SQL> ALTER TABLE agent DROP COLUMN commission_rate;

Table altered.
```

4. DELETE and UPDATE Queries

-- Delete all properties marked as 'Sold'
-- (ON DELETE CASCADE removes related SHOWING, OFFER and TRANSACTION rows)
DELETE FROM property WHERE status = 'Sold';

```
SQL> DELETE FROM property WHERE status = 'Sold';

2 rows deleted.
```

-- Reduce listing price by 5% for all 'Active' properties
UPDATE property
SET listing_price = listing_price * 0.95
WHERE status = 'Active';

```
SQL> UPDATE property
  2  SET listing_price = listing_price * 0.95
  3  WHERE status = 'Active';

2 rows updated.
```

-- Update the office location (office_name) of 'David Miller'
UPDATE agent
SET office_name = 'Downtown Luxury'
WHERE agent_name = 'David Miller';

```
SQL> UPDATE agent
  2  SET office_name = 'Downtown Luxury'
  3  WHERE agent_name = 'David Miller';

1 row updated.
```

-- Standard UPDATE queries for the remaining tables

UPDATE category
SET description = 'Independent houses, villas and townhouses'
WHERE category_id = 101;

```
SQL> UPDATE category
  2  SET description = 'Independent houses, villas and townhouses'
  3  WHERE category_id = 101;

1 row updated.
```

UPDATE client
SET phone = '9123456799'
WHERE client_id = 2002;

```
SQL> UPDATE client
  2  SET phone = '9123456799'
  3  WHERE client_id = 2002;

1 row updated.
```

UPDATE showing
SET showing_date = TO_DATE('16-AUG-2026','DD-MON-YYYY')
WHERE showing_id = 3005;

```
SQL> UPDATE showing
  2  SET showing_date = TO_DATE('16-AUG-2026','DD-MON-YYYY')
  3  WHERE showing_id = 3005;

1 row updated.
```

UPDATE offer
SET offer_status = 'Accepted'
WHERE offer_id = 4004;

```
SQL> UPDATE offer
  2  SET offer_status = 'Accepted'
  3  WHERE offer_id = 4004;

1 row updated.
```

UPDATE transaction
SET sale_price = sale_price + 5000
WHERE transaction_id = 5005;

```
SQL> UPDATE transaction
  2  SET sale_price = sale_price + 5000
  3  WHERE transaction_id = 5005;

1 row updated.
```

5. SELECT Queries

SELECT * FROM agent;

```
SQL> SELECT * FROM agent;

  AGENT_ID AGENT_NAME
---------- --------------------------------------------------
OFFICE_NAME
--------------------------------------------------
      9001 Sarah Jenkins
Downtown Luxury

      9002 David Miller
Downtown Luxury

      9003 Michael Chang
Coastal Prime


  AGENT_ID AGENT_NAME
---------- --------------------------------------------------
OFFICE_NAME
--------------------------------------------------
      9004 Amanda Ross
Downtown Luxury

      9005 Robert Vance
East Valley
```

SELECT * FROM property;

```
SQL> SELECT * FROM property;

PROPERTY_I
----------
ADDRESS
--------------------------------------------------------------------------------
LISTING_PRICE STATUS               CATEGORY_ID   AGENT_ID SQUARE_FEET
------------- -------------------- ----------- ---------- -----------
P5502
789 Ocean Blvd
      1187500 Active                       103       9003

P5503
456 Pine Ave
       380000 Pending                      101       9001

PROPERTY_I
----------
ADDRESS
--------------------------------------------------------------------------------
LISTING_PRICE STATUS               CATEGORY_ID   AGENT_ID SQUARE_FEET
------------- -------------------- ----------- ---------- -----------

P5505
555 Cedar Rd
       589000 Active                       101       9004
```

SELECT * FROM category;

```
SQL> SELECT * FROM category;

CATEGORY_ID CATEGORY_NAME
----------- ------------------------------
DESCRIPTION
--------------------------------------------------
        101 Residential
Independent houses, villas and townhouses

        102 Commercial
Office spaces and retail

        103 Luxury
High-end premium real estate


CATEGORY_ID CATEGORY_NAME
----------- ------------------------------
DESCRIPTION
--------------------------------------------------
        104 Land
Empty plots for development

        105 Industrial
Warehouses and factories
```

SELECT * FROM client;

```
SQL> SELECT * FROM client;

 CLIENT_ID CLIENT_NAME                                          PHONE
---------- -------------------------------------------------- ----------
CLIENT_TYPE
--------------------
      2001 John Smith                                          9123456780
Buyer

      2002 Emily Davis                                         9123456799
Seller

      2003 Mark Johnson                                        9123456782
Buyer


 CLIENT_ID CLIENT_NAME                                          PHONE
---------- -------------------------------------------------- ----------
CLIENT_TYPE
--------------------
      2004 Sophia Brown                                        9123456783
Investor

      2005 James Wilson                                        9123456784
Buyer
```

SELECT * FROM showing;

```
SQL> SELECT * FROM showing;

SHOWING_ID PROPERTY_I  CLIENT_ID SHOWING_D
---------- ---------- ---------- ---------
      3001 P5502            2001 10-AUG-26
      3002 P5503            2003 11-AUG-26
      3003 P5505            2005 12-AUG-26
      3004 P5502            2004 13-AUG-26
      3005 P5505            2001 16-AUG-26
```

SELECT * FROM offer;

```
SQL> SELECT * FROM offer;

  OFFER_ID PROPERTY_I  CLIENT_ID OFFER_AMOUNT OFFER_STATUS
---------- ---------- ---------- ------------ --------------------
      4001 P5502            2001      1200000 Rejected
      4002 P5503            2003       380000 Accepted
      4003 P5505            2005       600000 Pending
      4004 P5502            2004      1225000 Accepted
```

SELECT * FROM transaction;

```
SQL> SELECT * FROM transaction;

TRANSACTION_ID PROPERTY_I   OFFER_ID SALE_PRICE SALE_DATE
-------------- ---------- ---------- ---------- ---------
          5003 P5503            4002     380000 15-AUG-26
          5004 P5502            4004    1225000 20-AUG-26
          5005 P5505            4003     605000 25-AUG-26
```

Meaningful WHERE-conditioned queries:

SELECT address, listing_price FROM property WHERE status = 'Active';

```
SQL> SELECT address, listing_price FROM property WHERE status = 'Active';

ADDRESS
--------------------------------------------------------------------------------
LISTING_PRICE
-------------
789 Ocean Blvd
      1187500

555 Cedar Rd
       589000
```

SELECT client_name, phone FROM client WHERE client_type = 'Buyer';

```
SQL> SELECT client_name, phone FROM client WHERE client_type = 'Buyer';

CLIENT_NAME                                        PHONE
-------------------------------------------------- ---------------
John Smith                                         9123456780
Mark Johnson                                       9123456782
James Wilson                                       9123456784
```

SELECT offer_id, offer_amount FROM offer WHERE offer_status = 'Accepted';

```
SQL> SELECT offer_id, offer_amount FROM offer WHERE offer_status = 'Accepted';

  OFFER_ID OFFER_AMOUNT
---------- ------------
      4002       380000
      4004      1225000
```

6. Single Row Functions

SELECT UPPER(agent_name) AS agent_name_upper FROM agent;

```
SQL> SELECT UPPER(agent_name) AS agent_name_upper FROM agent;

AGENT_NAME_UPPER
--------------------------------------------------
SARAH JENKINS
DAVID MILLER
MICHAEL CHANG
AMANDA ROSS
ROBERT VANCE
```

SELECT LOWER(client_name) AS client_name_lower FROM client;

```
SQL> SELECT LOWER(client_name) AS client_name_lower FROM client;

CLIENT_NAME_LOWER
--------------------------------------------------
john smith
emily davis
mark johnson
sophia brown
james wilson
```

SELECT address, LENGTH(address) AS address_length FROM property;

```
SQL> SELECT address, LENGTH(address) AS address_length FROM property;

ADDRESS
--------------------------------------------------------------------------------
ADDRESS_LENGTH
--------------
789 Ocean Blvd
            14

456 Pine Ave
            12

555 Cedar Rd
            12
```

SELECT offer_id, ROUND(offer_amount) AS rounded_amount FROM offer;

```
SQL> SELECT offer_id, ROUND(offer_amount) AS rounded_amount FROM offer;

  OFFER_ID ROUNDED_AMOUNT
---------- --------------
      4001        1200000
      4002         380000
      4003         600000
      4004        1225000
```

SELECT INITCAP(category_name) AS category_title_case FROM category;

```
SQL> SELECT INITCAP(category_name) AS category_title_case FROM category;

CATEGORY_TITLE_CASE
------------------------------
Residential
Commercial
Luxury
Land
Industrial
```

7. Aggregate (Group) Functions

SELECT COUNT(*) AS total_properties FROM property;
SELECT SUM(listing_price) AS total_listing_value FROM property;
SELECT AVG(listing_price) AS average_listing_price FROM property;
SELECT MAX(offer_amount) AS highest_offer FROM offer;
SELECT MIN(offer_amount) AS lowest_offer FROM offer;

```
SQL> SELECT COUNT(*) AS total_properties FROM property;

TOTAL_PROPERTIES
----------------
               3

SQL> SELECT SUM(listing_price) AS total_listing_value FROM property;

TOTAL_LISTING_VALUE
-------------------
            2156500

SQL> SELECT AVG(listing_price) AS average_listing_price FROM property;

AVERAGE_LISTING_PRICE
---------------------
           718833.333

SQL> SELECT MAX(offer_amount) AS highest_offer FROM offer;

HIGHEST_OFFER
-------------
      1225000

SQL> SELECT MIN(offer_amount) AS lowest_offer FROM offer;

LOWEST_OFFER
------------
      380000
```

8. GROUP BY

SELECT status, COUNT(property_id) AS total_properties
FROM property
GROUP BY status;

SELECT agent_id, AVG(listing_price) AS avg_price
FROM property
GROUP BY agent_id;

```
SQL> SELECT status, COUNT(property_id) AS total_properties
  2  FROM property
  3  GROUP BY status;

STATUS               TOTAL_PROPERTIES
-------------------- ----------------
Active                              2
Pending                             1

SQL>
SQL> SELECT agent_id, AVG(listing_price) AS avg_price
  2  FROM property
  3  GROUP BY agent_id;

  AGENT_ID  AVG_PRICE
---------- ----------
      9003    1187500
      9001     380000
      9004     589000
```

9. HAVING

SELECT agent_id, COUNT(property_id) AS total_properties

FROM property
GROUP BY agent_id
HAVING COUNT(property_id) > 1;

```
SQL> SELECT agent_id, COUNT(property_id) AS total_properties
  2  FROM property
  3  GROUP BY agent_id
  4  HAVING COUNT(property_id) > 1;

no rows selected
```

10. Subquery

SELECT address, listing_price
FROM property
WHERE listing_price > (SELECT AVG(listing_price) FROM property);

```
SQL> SELECT address, listing_price
  2  FROM property
  3  WHERE listing_price > (SELECT AVG(listing_price) FROM property);

ADDRESS
--------------------------------------------------------------------------------
LISTING_PRICE
-------------
789 Ocean Blvd
      1187500
```

11. JOIN Queries

SELECT p.address, a.agent_name, p.listing_price
FROM property p
JOIN agent a ON p.agent_id = a.agent_id;

```
SQL> SELECT p.address, a.agent_name, p.listing_price
  2  FROM property p
  3  JOIN agent a ON p.agent_id = a.agent_id;

ADDRESS
--------------------------------------------------------------------------------
AGENT_NAME                                         LISTING_PRICE
-------------------------------------------------- -------------
456 Pine Ave
Sarah Jenkins                                           380000

789 Ocean Blvd
Michael Chang                                          1187500

555 Cedar Rd
Amanda Ross                                             589000
```

-- Comprehensive JOIN involving 3 tables: Property + Agent + Category
SELECT p.address, a.agent_name, c.category_name, p.listing_price
FROM property p
JOIN agent a ON p.agent_id = a.agent_id
JOIN category c ON p.category_id = c.category_id;

```
SQL> SELECT p.address, a.agent_name, c.category_name, p.listing_price
  2  FROM property p
  3  JOIN agent a ON p.agent_id = a.agent_id
  4  JOIN category c ON p.category_id = c.category_id;

ADDRESS
--------------------------------------------------------------------------------
AGENT_NAME
--------------------------------------------------
CATEGORY_NAME                  LISTING_PRICE
------------------------------ -------------
555 Cedar Rd
Amanda Ross
Residential                          589000

456 Pine Ave
Sarah Jenkins
Residential                          380000

ADDRESS
--------------------------------------------------------------------------------
AGENT_NAME
--------------------------------------------------
CATEGORY_NAME                  LISTING_PRICE
------------------------------ -------------

789 Ocean Blvd
Michael Chang
Luxury                              1187500
```

12. CONCLUSION

This project successfully designed, normalised, implemented, and validated a relational database for a Real Estate Sales Management System using Oracle Database. All seven core entities (AGENT, PROPERTY, CATEGORY, CLIENT, SHOWING, OFFER, TRANSACTION) along with the APP_USERS authentication entity were constructed with strict primary key, foreign key, NOT NULL, UNIQUE, and CHECK constraints. The complete relational lifecycle was demonstrated: structured data insertion, schema evolution via ALTER TABLE, cascading deletions via ON DELETE CASCADE, multi-table JOIN operations, scalar single-row functions, aggregate statistics, GROUP BY and HAVING clauses, and subqueries. The database was seamlessly wrapped in a full-stack Electron desktop application with Role-Based Access Control and analytical dashboards, establishing an efficient, consistent, and scalable operational platform.

13. REFERENCES

1. Oracle Corporation, "Oracle Database SQL Language Reference", Oracle Help Center Documentation.
2. Database Management System (DBMS) Course Materials, BACSE202 Lab Component, VIT, 2026-27.
3. Oracle Corporation, "Oracle SQL*Plus User's Guide and Reference", Oracle Database Documentation Library.
4. Elmasri, R. and Navathe, S. B., "Fundamentals of Database Systems", 7th Edition, Pearson Education.
5. Silberschatz, A., Korth, H. F. and Sudarshan, S., "Database System Concepts", 7th Edition, McGraw-Hill.